Technical Specification — Context-Aware Q&A Assistant

Version	1.0
Date	2026-10-26
Related	Functional specification · Architecture · Prompt design

1. Technology stack

Layer	Choice	Why
Language	Python 3.10+ (developed on 3.14)	Standard for AI tooling.
LLM	Google Gemini (`GEMINI_MODEL` , default `gemini-flash-latest`) via the official `google-genai` Python SDK	System instruction plus JSON-schema output for grounded, machine-readable answers.
Retrieval	BM25 keyword search implemented in pure Python	Knowledge base is small (≈50 sections); no vector DB or embedding service needed; fully explainable scores.
Web	FastAPI + Uvicorn, one static HTML page (vanilla JS)	Small, typed, auto-generated OpenAPI docs at `/docs` .
Config	Environment variables / `.env` via `python-dotenv`	Keeps secrets out of code.
Tests	pytest + FastAPI `TestClient` ; Gemini client is faked	Tests run offline, no API cost.

2. Module layout

```
qa_assistant/
├── config.py          Settings dataclass; env vars; mode resolution (auto → llm | offline)
├── knowledge_base.py  Chunk dataclass; Markdown loader and section splitter
├── retriever.py       Tokenizer, stemmer, synonym map, BM25Retriever
├── prompts.py         SYSTEM_PROMPT, ANSWER_SCHEMA, context formatting
├── answerers.py       GeminiAnswerer (LLM), ExtractiveAnswerer (offline), Draft, AnswerError
├── gemini_models.py   Lists the Gemini models the API key can use
├── pipeline.py        QAAssistant.ask(): validate → retrieve → answer → verify citations
├── api.py             FastAPI app: /, /api/ask, /api/health, /api/documents
├── cli.py             Command-line interface
└── evaluate.py        Evaluation runner and report writer
```

3. Data model

```
Chunk(id, source, title, section, text)          # id = "<file stem>#<section slug>"
RetrievedChunk(chunk: Chunk, score: float)
Draft(status, answer, citations: list[str], missing_information)   # answerer output
Answer(question, status, answer, citations: list[Source], missing_information,
       retrieved: list[Source], mode, latency_ms, notes)            # pipeline output
Source(id, source, section, score, text)
```

`status` ∈ {`answered`, `partial`, `not_found`}.

4. Processing pipeline

4.1 Ingestion (start-up)

1. Read every `*.md` / `*.txt` in `QA_KB_DIR` (sorted for determinism).
2. `#` heading → document title. Each `##` heading starts a new chunk; text before the first `##` becomes the "Overview" chunk. Empty chunks are dropped.
3. Build the BM25 index. Indexed text = title + section heading (twice, to boost heading matches) + body.

4.2 Tokenization

- Lowercase; tokens match `[a-z0-9]+` with internal `.` `:` `,` between digits kept (`10:00`, `1,500` → `1500`).
- English stopwords removed (including question words like *how many*).
- Grade letters are kept: *Band A* → `banda` (otherwise *a* would be dropped as a stopword).
- Light suffix stemmer (`ies/ied→y`, `ing`, `ed`, `es`, `s`) so *days/day*, *carried/carry* match.
- Query-side synonym expansion (e.g. *vacation → leave, annual*; *WFH → remote, home*; *PF/EPF → provident fund*; *mileage → km, conveyance*).

4.2a Spelling correction

Before searching, each unknown word of 5+ letters is replaced by the closest knowledge-base word (`difflib`, similarity ≥ 0.8) **only if it starts with the same letter** — *internatioal* → *international*, *hotal* → *hotel*, but *relocation* is not turned into *location*, so real-but-uncovered topics stay uncovered. Corrections are reported in the answer's `notes` ("Searched for 'international' (you typed 'internatioal')"). Short forms are handled by synonyms: *dem / perdiem / DA* → *diem*, *trip* → *travel*, *hotel / accommodation* → *stay*. Gemini receives the original wording; the offline answerer uses the corrected one.

4.3 Retrieval — BM25

`score(q, d) = Σ idf(t) · tf(t,d)·(k1+1) / (tf(t,d) + k1·(1 − b + b·|d|/avgdl))`, with `k1 = 1.5`, `b = 0.75`, `idf(t) = ln(1 + (N − df + 0.5)/(df + 0.5))`. Only chunks with score > 0 are returned; top `QA_TOP_K` (default 5). A chunk whose section heading contains every word of the question (e.g. the question *International Travel: Per Diem*) gets its score ×1.5.

Short-circuit: if nothing scores above zero, the pipeline returns `not_found` without calling the model (saves cost on clearly off-topic questions such as "Who won the cricket World Cup?").

4.4 Answer generation — Gemini mode (`GeminiAnswerer`)

One call to `client.models.generate_content` (`google-genai` SDK; `genai.Client()` reads `GEMINI_API_KEY` or `GOOGLE_API_KEY`):

Parameter	Value
`model`	`GEMINI_MODEL`, default `gemini-flash-latest` (alias that follows Google's current Flash model)
`contents`	`<context><document id=… source=… section=…>…</document>…</context>` + `<question>…</question>`
`config.system_instruction`	`SYSTEM_PROMPT` (grounding rules — see prompt_design.md)
`config.response_mime_type` / `response_json_schema`	`application/json` / `ANSWER_SCHEMA` (status enum, answer, citations[], missing_information) — guarantees parseable output
`config.max_output_tokens`	16000

Response handling: finish reason `SAFETY`, `PROHIBITED_CONTENT`, `BLOCKLIST` or `SPII` → `not_found`; `MAX_TOKENS` → error; otherwise `json.loads(response.text)`. Errors: invalid key (Google returns 400 "API key not valid", or 401/403) → "check GEMINI_API_KEY"; 404 → model not found, with a pointer to the model list; 429 → quota/rate limit; others → code and message. All become HTTP 502 in the API.

`python -m qa_assistant.gemini_models` lists the models the key can use (`client.models.list()`, filtered to those supporting `generateContent`), so `GEMINI_MODEL` can be pinned to a specific version.

4.5 Answer generation — offline mode (`ExtractiveAnswerer`)

For demos and CI without an API key:

1. Compute **IDF-weighted term coverage** of the question by the top chunk (a question word counts as covered if it or a synonym appears). Rare words like *stock* weigh more than *employee*.
2. Coverage < 0.65 → `not_found` with the reason. Numbers in the question that the section does not contain are ignored (they are values to reason about, e.g. *a 4-hour flight*). The 0.65 threshold was calibrated on the evaluation set, so treat offline results as optimistic.
3. Otherwise return the 2 sentences (from the top 2 chunks) with the highest IDF-weighted query-term overlap (a rare match such as *Band B* outranks several common ones such as *travel*), citing their chunks. Markdown table rows are turned into sentences first (`| Band A | AC 2-tier |` → *Band A: Train class AC 2-tier.*).

Two refinements: (a) a **topic question** that names no band and whose words match a section heading (heading coverage ≥ 0.65, best match among the top 3 sections that contain a table) is answered with that section's **whole table** plus its introductory sentence — *per diem for international trip* returns all three country groups; (b) a question that names a band (*Band C*) is answered only from sentences or table rows that mention that band.

It cannot reason (e.g. "₹1,00,000 → department head") — it is a baseline, not the main mode.

4.6 Post-processing (`QAAssistant._finalize`)

- Citations not present in the retrieved set are dropped and reported in `notes` (guards against invented sources).
- A non-`not_found` answer with no valid citation gets a caution note.

5. Interfaces

5.1 HTTP API

Method	Path	Body / response
`POST`	`/api/ask`	`{"question": "..."}` → `Answer` JSON. 422 invalid question, 502 model error.
`GET`	`/api/health`	`{"status","mode","model","chunks","documents"}`
`GET`	`/api/documents`	list of documents and their section ids
`GET`	`/`	Web UI. `/?q=...` auto-asks a question.
`GET`	`/docs`	Swagger UI (FastAPI)

Example response:

```
{
  "question": "Can employees carry forward more than 10 days of leave?",
  "status": "answered",
  "answer": "No. Employees may carry forward at most 10 days ...",
  "citations": [{"id": "leave_policy#carry-forward-of-annual-leave", "source":
"leave_policy.md",
                 "section": "Carry Forward of Annual Leave", "score": 17.401, "text": "..."}],
  "missing_information": "",
  "retrieved": [ ... 5 sources ... ],
  "mode": "gemini",
  "latency_ms": 2140,
  "notes": []
}
```

5.2 CLI

`python -m qa_assistant.cli [--mode auto|llm|offline] [--show-context] [question...]`

5.3 Evaluation

`python -m qa_assistant.evaluate [--mode auto|llm|offline]` → `eval/results/results_<mode>.json` and `.md`.

6. Configuration

Variable	Default	Meaning
`GEMINI_API_KEY`	–	Gemini API key (from Google AI Studio). `GOOGLE_API_KEY` also works.
`GEMINI_MODEL`	`gemini-flash-latest`	Gemini model id; list options with `python -m qa_assistant.gemini_models`
`QA_MODE`	`auto`	`auto` (Gemini if `GEMINI_API_KEY` is set, else offline), `gemini`, `offline`
`QA_TOP_K`	`5`	Sections passed to the model
`QA_KB_DIR`	`knowledge_base`	Folder with policy files

7. Evaluation method

`eval/questions.json` holds 48 items: `id`, `category` (`answerable` / `unanswerable`), `question`, `expected_source`, `expected_keywords` (list of groups; each group is a list of acceptable alternatives, all groups must match, word-boundary, case-insensitive).

Metric	Definition
Answerable accuracy	status ≠ not_found **and** all keyword groups found in the answer
Unanswerable decline rate	status == not_found on unanswerable questions
Hallucination rate	status answered/partial on unanswerable questions
Wrongly declined	answerable questions returned as not_found
Retrieval hit rate	expected source document among retrieved sections

Keyword grading is deterministic and free; its limitation is that it checks facts, not phrasing quality.

8. Security considerations

- Key only from env/`.env`; `.env` is git-ignored.
- Question length capped at 1000 chars (API and pipeline).
- Prompt-injection resistance: documents and question are wrapped in tags and declared data; system prompt forbids following embedded instructions; tested by eval item U12.
- UI escapes all model output before inserting it into HTML.

9. Testing

`pytest` (22 tests): Gemini request shape (system prompt, schema, context), Gemini safety blocks, citation filtering, model-call short-circuit, auto-mode selection, spelling correction, whole-table answers, chunking, stemming, band-letter tokens, table-to-sentence conversion, retrieval ranking, offline answers and declines, HTTP endpoints.

10. Known limitations and future work

- Lexical retrieval misses paraphrases that share no words with the policy (partly mitigated by synonyms). Upgrade path: hybrid BM25 + embeddings.
- No conversation memory; follow-ups must be self-contained.
- Knowledge base is loaded at start-up; restart to pick up edits.
- Larger corpora: add prompt caching for a stable system prompt, and chunk long sections.